

Semestrální práce KIV/UPG

Tento kód je návrhem grafické aplikace ve WinForms, která simuluje elektromagnetické pole. Soubor funkcí byl navržen pro simulaci interakce mezi magnety a sondou.

Spuštění programu:

- Program se přeloží pomocí příkazu `.\Build.cmd` v adresáři souboru
- Program pak lze spustit pomocí příkazu `.\Run.cmd` v adresáři souboru
- Za příkaz `.\Run.cmd` mohou být zapsány maximálně 2 parametry:
 - 1. Parametr – udává jaký scénář bude vykreslit 0 - 4
 - 2. Parametr – udává šířku a výšku vzorku mřížky a to ve formátu `-g<X>x<Y>`, kde X je zadávaná šířka a Y je zadávaná výška

Uživatelská příručka

Ovládání aplikace:

- Spuštění a základní ovládání:
- V horní části okna najdete panel s ovládacími prvky
- ComboBox pro výběr scénáře (1-4)
- Tlačítka pro ovládání rychlosti sondy a magnetů
- Tlačítka pro přidání a odebrání magnetů

Práce s magnety:

- Přidání magnetu: tlačítko "Add Magnet"
- Odebrání magnetu: vyberte magnet kliknutím a stiskněte "Remove Magnet"
- Přesouvání magnetů: uchopte magnet myší a přetáhněte na novou pozici
- Změna síly magnetu: najedte na magnet a použijte kolečko myši
- Polarita magnetu se mění automaticky při snížení síly pod minimum
- Pro dynamické magnety lze měnit rychlost změny síly tlačítkem "Magnet Change Speed"(0.5x, 1x, 2x)

Práce se sondou:

- Kliknutím do prostoru umístíte statickou sondu
- Po umístění sondy se otevře graf měření intenzity
- Rychlost pohybu sondy lze měnit tlačítkem "Probe Speed" (0.5x, 1x, 2x)

Další funkce:

- Export do SVG: tlačítko "Export to SVG"
- Návrat do výchozího stavu: tlačítko "Back to Default"
- Změna scénáře: výběr z ComboBoxu "Change Scenario"

Vizualizace:

- Červená barva označuje kladnou polaritu magnetu
- Modrá barva označuje zápornou polaritu magnetu
- Šipky v mřížce ukazují směr a velikost intenzity pole
- Teplotní mapa zobrazuje intenzitu pole barevným gradientem
- Graf zobrazuje změny intenzity v čase pro statickou sondu

Postup řešení:

1. Vypočítáme směrový vektor mezi sondou a magnetem (nebo nábojem) — tento vektor ukazuje směr, kterým magnet působí na sondu.
2. Vydělíme směrový vektor vzdáleností mezi magnetem a sondou na třetí — tím zohledníme, že síla klesá s třetí mocninou vzdálenosti (to je dáno Coulombovým zákonem pro elektrická pole nebo zákonem pro magnetická pole).
3. Vynásobíme silou (nábojem nebo magnetickou silou) magnetu — toto zohledňuje, jak silný je daný magnet (náboj). Výsledkem bude vektor síly (nebo pole), který má správnou velikost i směr.
4. Tento výpočet provedeme pro všechny magnety — pole nebo síla od každého magnetu se počítá zvlášť.
5. Sčítáme výsledné vektory — protože síly působí současně od více magnetů, je třeba je vektorově sečíst (v každém směru: x , y , z).
6. Vypočítáme velikost výsledného vektoru — výsledný vektor je součet všech jednotlivých sil, a pokud chceme znát celkovou velikost síly, spočítáme délku tohoto výsledného vektoru.
7. Výsledek je síla (nebo celková hodnota pole) v bodě, kde je umístěna sonda.

V následující části bude popsán každý soubor který se nachází v řešení:

Program.cs

Soubor: Program.cs

Popis:

Tento soubor obsahuje hlavní vstupní bod pro aplikaci. Zajišťuje zpracování argumentů příkazového řádku a spouští hlavní formulář aplikace s odpovídajícími parametry.

Funkce:

- `Main(string[] args)`: Hlavní metoda aplikace, která zpracovává argumenty příkazového řádku a spouští aplikaci s odpovídajícími parametry.

Detaily:

- `Regex regex = new Regex("[0-9]+$")`: Vytvoří regulární výraz pro kontrolu, zda řetězec obsahuje pouze číslice.
- `Application.EnableVisualStyles()`: Povolení vizuálních stylů pro aplikaci.
- `Application.SetCompatibleTextRenderingDefault(false)`: Nastavení kompatibilního vykreslování textu na `false`.
- Zpracování argumentů:
- `args.Length == 0`: Pokud nejsou předány žádné argumenty, aplikace se spustí s výchozími parametry (`MainForm(0, 0, 0)`).
- `args.Length == 1`: Pokud je předán jeden argument, zkontroluje se, zda je to platné celé číslo mezi 0 a 4. Pokud ano, aplikace se spustí s tímto parametrem.
- `args.Length == 2`: Pokud jsou předány dva argumenty, zkontroluje se, zda první argument obsahuje pouze číslice. Druhý argument se rozdělí na části X a Y, které se použijí jako parametry pro spuštění aplikace.
- `args.Length > 2`: Pokud je předáno více než dva argumenty, zobrazí se chybová zpráva.

Chybové zprávy:

Pokud je předán neplatný parametr, zobrazí se chybová zpráva s odpovídajícím popisem problému.

MainForm.cs

Soubor: MainForm.cs

Popis:

Tento soubor obsahuje třídu MainForm, která je hlavním formulářem aplikace. Formulář zobrazuje magnetické pole a umožňuje interakci s magnety.

Funkce:

- MainForm(int scenario, int spacing_x, int spacing_y): Konstruktor, který inicializuje formulář s daným scénářem a rozestupy mřížky.
- drawingPanel_Resize(object sender, EventArgs e): Metoda, která se volá při změně velikosti panelu a obnovuje jeho vykreslení.
- drawingPanel_Paint(object sender, PaintEventArgs e): Metoda, která vykresluje obsah panelu, včetně magnetů, mřížky a sondy.
- EnableDoubleBuffering(Control control): Povolení dvojitého bufferování pro plynulejší vykreslování.
- Timer_Tick(object sender, EventArgs e): Metoda, která se volá při každém tiknutí časovače a aktualizuje stav magnetů a sondy.

Správa scénářů:

- ScenarioComboBox_SelectedIndexChanged(object sender, EventArgs e): Metoda pro přepínání mezi scénáři, resetuje příslušné stavy a aktualizuje zobrazení.
- Správa magnetů:
- add_magnet_Click(object sender, EventArgs e): Přidává nový magnet do simulace.
- remove_magnet_Click(object sender, EventArgs e): Odstraňuje vybraný magnet ze simulace.
- back_default_Click(object sender, EventArgs e): Vrací scénář do výchozího stavu.
- Ovládání rychlosti:
- button_speed_Click(object sender, EventArgs e): Ovládá rychlost pohybu sondy (0.5x, 1x, 2x).
- button_speed_magnet_Click(object sender, EventArgs e): Ovládá rychlost změny síly magnetů v dynamickém režimu.

Export:

- `exportSvgButton_Click(object sender, EventArgs e)`: Exportuje aktuální zobrazení do SVG formátu.
- Interakce s myší:
- `drawingPanel_MouseClick(object sender, MouseEventArgs e)`: Umisťuje statickou sondu.
- `drawingPanel_MouseDown(object sender, MouseEventArgs e)`: Zahajuje tažení magnetu.
- `drawingPanel_MouseMove(object sender, MouseEventArgs e)`: Aktualizuje pozici taženého magnetu.
- `drawingPanel_MouseUp(object sender, MouseEventArgs e)`: Ukončuje tažení magnetu.
- `drawingPanel_MouseWheel(object sender, MouseEventArgs e)`: Mění sílu magnetu.
- `checkIfMouselsOverMagnet(PointF mousePosition, Magnet magnet)`: Kontroluje, zda je kurzor nad magnetem.

Detaily:

- `scale`: Měřítko pro vykreslování
- `DynamicMagnets`: Určuje, zda jsou magnety dynamické
- `StaticProbe`: Určuje, zda je sonda statická
- `intensityValues` a `timeValues`: Seznamy pro ukládání hodnot intenzity a času pro graf
- `chartForm`: Formulář pro zobrazení grafu
- `isDragging`: Určuje, zda je magnet tažen myší
- `mouseOffset`: Posun myši při tažení magnetu
- `selectedMagnet`: Aktuálně vybraný magnet
- `lastClickedMagnet`: Poslední kliknutý magnet

Magnet.cs

Soubor: Magnet.cs

Popis:

Tento soubor obsahuje definici třídy Magnet, která reprezentuje magnet s určitými vlastnostmi, jako jsou souřadnice, polarita a síla.

Třída

Magnet: Třída reprezentující magnet.

Atributy

- float X: Souřadnice X magnetu.
- float Y: Souřadnice Y magnetu.
- string polarity: Polarita magnetu, může být "+" nebo "-".
- float force: Síla magnetu.
- Konstruktor
- Magnet(float X, float Y, string polarity, float force): Inicializuje novou instanci třídy Magnet s danými souřadnicemi, polaritou a silou.

MagnetLogic.cs

Soubor: MagnetLogic.cs

Popis:

Tento soubor obsahuje logiku pro manipulaci s magnety v aplikaci. Obsahuje metody pro změnu síly magnetů a výběr scénáře, který inicializuje magnety s odpovídajícími parametry.

Funkce:

- `ChangeMagnetsForce(Magnet[] magnets)`: Metoda pro změnu síly magnetů na základě aktuálního času.
- `ChooseScenario(int scenario, Boolean DynamicMagnets, MainForm main)`: Metoda pro výběr scénáře a inicializaci magnetů.

Detaily:

- `float t = Environment.TickCount / 1000f`: Získání aktuálního času v sekundách.
- `float q1 = 1 + 0.5f * (float)Math.Sin(Math.PI / 2 * t)`: Výpočet nové síly pro první magnet.
- `float q2 = 1 - 0.5f * (float)Math.Sin(Math.PI / 2 * t)`: Výpočet nové síly pro druhý magnet.
- `magnets[0].force = q1`: Nastavení nové síly pro první magnet.
- `magnets[1].force = q2`: Nastavení nové síly pro druhý magnet.

Scénáře:

- `scenario == 0`: Inicializuje jeden magnet s polaritou "+" a silou 1.
- `scenario == 1`: Inicializuje dva magnety s polaritou "+" a silou 1, umístěné na souřadnicích (-1, 0) a (1, 0).
- `scenario == 2`: Inicializuje dva magnety s polaritami "-" a "+", a silami 1 a 2, umístěné na souřadnicích (-1, 0) a (1, 0).
- `scenario == 3`: Inicializuje čtyři magnety s různými polaritami a silami, umístěné na souřadnicích (-1, -1), (1, -1), (1, 1) a (-1, 1).
- `scenario == 4`: Aktivuje dynamické magnety a inicializuje dva magnety s polaritou "+" a silou 1, umístěné na souřadnicích (-1, 0) a (1, 0).

DrawingFunctions.cs

Soubor: DrawingFunctions.cs

Popis:

Tento soubor obsahuje funkce pro kreslení různých prvků na panelu v aplikaci. Obsahuje metody pro kreslení magnetů, teplotní mapy, legendy, souřadnicového systému, mřížky, vektorů a sond.

Funkce:

- DrawMagnets(Graphics g, Magnet[] magnets): Metoda pro vykreslení všech magnetů.
- DrawMagnet(Graphics g, float X, float Y, string polarity, float force): Metoda pro vykreslení jednoho magnetu.
- DrawHeatMap(Graphics g, in Magnet[] magnets): Metoda pro vykreslení teplotní mapy.
- DrawLegend(Graphics g, float probeSpeed, float magnetSpeed, int currentScenario): Metoda pro vykreslení legendy.
- DrawCoordinateSystem(Graphics g, int space_x, int space_y): Metoda pro vykreslení barevná mapy reprezentující velikosti intenzity.
- DrawGrid(Graphics g, Magnet[] magnets, int space_x, int space_y): Metoda pro vykreslení mřížky a intenzitních vektorů.
- DrawArrowInCell(Graphics g, float cellCenterX, float cellCenterY, PointF vector, int a_size): Metoda pro vykreslení šipky v buňce mřížky.
- DrawVector(Graphics g, PointF start, PointF end, double description, int c): Metoda pro vykreslení vektoru s popisem.
- DrawCircularProbe(Graphics g): Metoda pro vykreslení kruhové sondy.
- DrawStaticProbe(Graphics g, float x, float y, float probe_size): Metoda pro vykreslení statické sondy.
- DrawCoordinates(Graphics g, PointF point): Metoda pro vykreslení souřadnic sondy.

Detaily:

- float size_R = 30 * scale / 100 + 20 * force: Výpočet velikosti magnetu na základě měřítka a síly.
- Color startColor = magnet.polarity == "+" ? Color.Red : Color.Blue: Nastavení barvy teplotní mapy na základě polarity magnetu.
- g.TranslateTransform(-drawingPanel.Width / 2 + 10, -drawingPanel.Height / 2 + 10): Posunutí souřadnicového systému pro vykreslení legendy.
- float elapsed = (Environment.TickCount - (int)m_startTime) / 1000f: Výpočet uplynulého času od startu.

Chybové zprávy:

Pokud jsou předány neplatné parametry pro vykreslení, metoda se pokusí vykreslit prvek s výchozími hodnotami nebo neprovede žádnou akci.

ChartForm.cs

Soubor: ChartForm.cs

Popis:

Tento soubor obsahuje třídu ChartForm, která je zodpovědná za zobrazení grafu intenzity v čase pomocí knihovny LiveChartsCore. Obsahuje metody pro inicializaci grafu a aktualizaci dat v grafu.

Funkce:

- ChartForm(List values, List time): Konstruktor třídy, který inicializuje komponenty formuláře, inicializuje graf a aktualizuje graf s poskytnutými hodnotami intenzity a času.
- InitializeChart(): Metoda pro inicializaci grafu, nastavení jeho vlastností a přidání do formuláře.
- UpdateChart(List intensityValues, List timeValues): Metoda pro aktualizaci grafu s novými hodnotami intenzity a času.

Detaily:

- cartesianChart = new CartesianChart { Dock = DockStyle.Fill, ZoomMode = LiveChartsCore.Measure.ZoomAndPanMode.X }; Inicializace grafu a nastavení jeho vlastností, jako je ukotvení a režim přiblížení/posunu.
- this.Controls.Add(cartesianChart); Přidání grafu do formuláře.
- this.Text = "Intensity Over Time"; Nastavení textu titulku formuláře.
- this.Size = new Size(600, 400); Nastavení velikosti formuláře.
- List scaledIntensityValues = intensityValues.ConvertAll(value => value / 1_000_000); Převod hodnot intenzity na miliony Newtonů na Coulomb.
- var lineSeries = new LineSeries { ... }; Vytvoření nové řady pro graf s nastavením vlastností, jako je barva, tloušťka čáry a velikost geometrie.
- cartesianChart.Series = new ISeries[] { lineSeries }; Nastavení řady pro graf.
- cartesianChart.XAxes = new Axis[] { ... }; Nastavení osy X grafu s popisky a rychlostí animace.
- cartesianChart.YAxes = new Axis[] { ... }; Nastavení osy Y grafu s popisky a rychlostí animace.

Chybové zprávy:

Pokud jsou předány neplatné nebo prázdné seznamy hodnot intenzity nebo času, graf nebude aktualizován správně.

CalculationsFunctions.cs

Soubor: CalculationsFunctions.cs

Popis:

Tento soubor obsahuje třídu CalculationsFunctions, která poskytuje metody pro výpočet intenzity elektrického pole, vzdálenosti a směrových vektorů mezi magnety a sondou. Obsahuje také metody pro normalizaci vektorů a součet vektorů.

Funkce:

- `CountIntensityVector(Magnet[] magnets, PointF probe)`: Metoda pro výpočet intenzitního vektoru v daném bodě (probe) na základě pole magnetů.
- `CountDistance(Magnet magnet, PointF probe)`: Metoda pro výpočet vzdálenosti mezi magnetem a sondou.
- `CountDirectionVector(Magnet magnet, PointF probe)`: Metoda pro výpočet směrového vektoru mezi magnetem a sondou.
- `VectorsSum(PointF[] vectors)`: Metoda pro součet pole vektorů.
- `NormalizeVector(PointF vector)`: Metoda pro normalizaci vektoru.
- `CountIntensity(PointF IntensityVector)`: Metoda pro výpočet intenzity elektrického pole na základě intenzitního vektoru.

Detaily:

- `PointF[] vectors = new PointF[magnets.Length];` Inicializace pole vektorů pro uložení intenzitních vektorů jednotlivých magnetů.
- `double distance = CountDistance(magnet, probe);` Výpočet vzdálenosti mezi magnetem a sondou.
- `PointF vector = CountDirectionVector(magnet, probe);` Výpočet směrového vektoru mezi magnetem a sondou.
- `PointF intensity_vector = new PointF(magnet.force * (vector.X / (float)Math.Pow(distance, 3)), magnet.force * (vector.Y / (float)Math.Pow(distance, 3)));` Výpočet intenzitního vektoru na základě síly magnetu a vzdálenosti.
- `PointF Intensity_vector_sum = VectorsSum(vectors);` Součet všech intenzitních vektorů.
- `double distance = Math.Sqrt(Math.Pow(magnet.X - probe.X, 2) + Math.Pow(magnet.Y - probe.Y, 2));` Výpočet vzdálenosti pomocí Pythagorovy věty.
- `if (magnet.polarity == "+") { ... } else { ... }` Podmínka pro určení směru vektoru na základě polarity magnetu.
- `PointF vector_normalized = new PointF((float)(vector.X / vector_len), (float)(vector.Y / vector_len));` Normalizace vektoru na jednotkovou délku.
- `double q = 1 / (4 * (float)Math.PI * e0);` Výpočet konstanty pro intenzitu elektrického pole.

Chybové zprávy:

Pokud jsou předány neplatné nebo prázdné hodnoty pro magnety nebo sondy, metody mohou vrátit nesprávné výsledky nebo vyvolat výjimky.

Závěr:

Aplikace úspěšně implementuje simulaci elektromagnetického pole s interaktivními prvky. Využívá objektově orientovaný přístup s jasně oddělenými třídami pro výpočty, vykreslování a logiku magnetů. Uživatelské rozhraní poskytuje intuitivní ovládání a různé možnosti vizualizace pole.

Hlavní dosažené cíle:

- Implementace fyzikálního modelu elektromagnetického pole
- Interaktivní manipulace s magnety včetně změny síly a polarity
- Vizualizace pole pomocí vektorů a teplotní mapy
- Měření intenzity pole pomocí pohyblivé a statické sondy
- Export vizualizace do SVG formátu
- Podpora různých přednastavených scénářů